

Chapter 18 作业

学号：23336128

姓名：梁力航

问题1：证明两阶段锁协议保证冲突可串行化

证明两阶段锁协议(Two-Phase Locking Protocol)能够保证冲突可串行化，且事务可以根据它们的锁点进行串行化。

回答：

两阶段锁协议 (2PL) 的定义：每个事务分为两个阶段：增长阶段可以获取锁但不能释放锁，缩减阶段可以释放锁但不能获取新锁。

证明思路：通过构造事务的优先级图，证明2PL下的任何调度都是冲突可串行化的。

关键概念：锁点 (Lock Point)：事务释放第一个锁的时刻。

证明过程：假设存在两个事务 T1 和 T2，它们的锁点分别为 L1 和 L2，且 $L1 < L2$ （即 T1 先进入缩减阶段）。在2PL下，T1 在 L1 之后不再获取新锁，而 T2 在 L2 之前仍处于增长阶段（因为 L2 是 T2 的第一个释放锁的时刻）。考虑 T1 和 T2 之间的冲突操作，如果 T1 的操作先于 T2 的操作，则优先级图中有 $T1 \rightarrow T2$ 的边；如果 T2 的操作先于 T1 的操作，这意味着 T2 在 T1 释放锁之前就获取了冲突锁，但根据2PL的增长阶段规则，T1 在 L1 之前处于增长阶段，而 T2 在 L2 之前也处于增长阶段，由于 $L1 < L2$ ，T1 必须在 T2 之前完成所有锁的获取，因此这种情况不可能发生。因此，所有冲突操作的顺序都与锁点的顺序一致，即 $T1 \rightarrow T2$ 。推广到多个事务，所有事务的优先级图是一个有向无环图 (DAG)，其中边的方向由锁点的顺序决定。根据冲突可串行化的定义，优先级图为DAG的调度是冲突可串行化的。

结论：两阶段锁协议保证了冲突可串行化，且事务可以根据它们的锁点顺序进行串行化。

问题2：数据库锁层次的锁获取策略

假设数据库的锁层次包括数据库、关系和元组。

回答：

a. 如果一个事务需要从关系 r 中读取大量元组，它应该获取哪些锁？

事务应该获取关系 r 的共享锁 (S锁)。这样可以保证事务读取关系 r 中的所有元组时，没有其他事务能修改其中任何元组，同时减少锁表开销（只需一个关系级锁而非每个元组单独的锁），并降低潜在的死锁风险。

b. 现在假设事务在读取了大量元组后想要更新 r 中的一些元组。它应该获取哪些锁？

事务有两种主要选择：首先将关系 r 的共享锁（S锁）升级为排他锁（X锁），或直接获取关系 r 的排他锁（若事务开始时就知道需要更新）；若更新的元组数量较少，也可获取关系 r 的意向排他锁（IX锁），再为每个需更新的元组获取排他锁（X锁）。选择策略取决于更新比例，若超过20%等较大比例，直接升级为关系级X锁更高效；若比例较小，使用IX锁加元组X锁能提高并发性。

c. 如果在运行时事务发现需要实际更新大量元组（在获取锁时假设只有少量元组会被更新），这将对锁表造成什么问题？数据库可以采取什么措施来避免这个问题？

问题：为每个元组获取单独的锁会导致锁表膨胀，消耗大量内存；同时大量元组锁会增加与其他事务的冲突概率，降低并发性，也提高了死锁风险。

解决措施：数据库可采用锁升级机制，当事务获取的元组锁数量超过阈值时自动升级为更高级别锁（如关系锁）；使用多粒度锁定和意向锁机制，允许在粗粒度上获取锁的同时支持细粒度锁定；通过自动锁管理根据事务操作模式调整锁粒度，优化锁表使用；或要求事务预声明锁粒度，避免运行时的锁升级问题。

问题3：根树组织的数据库中树协议的并发性

考虑一个以根树(rooted tree)形式组织的数据库。假设在每对顶点之间插入一个虚拟顶点。说明，如果在新树上遵循树协议（Tree Protocol），我们将获得比在原始树上遵循树协议更好的并发性。

回答：

树协议（Tree Protocol）的核心规则：事务可首先锁定根节点，要锁定一个节点必须已锁定其父节点，可以随时释放锁无需遵循两阶段锁协议，但不能重新锁定已释放的节点。

原始树与新树的结构差异：原始树中节点直接连接，如 $A \rightarrow B \rightarrow C$ ；新树则在每对节点间插入虚拟节点，如 $A \rightarrow A-B \rightarrow B \rightarrow B-C \rightarrow C$ 。

并发性提高的原因：虚拟节点的引入带来更细粒度的锁定控制，可更精确地控制数据访问范围；同时允许更早释放锁，事务处理完一个节点后即可释放虚拟节点的锁，无需等待整个分支处理完成；虚拟节点的存在还减少了锁冲突，使不同事务可同时锁定不同虚拟节点，并行访问原始树中的相邻节点。例如，原始树中若事务 T_1 锁定 $A \rightarrow B \rightarrow C$ ，事务 T_2 必须等待 T_1 释放 B 才能访问 $B \rightarrow D$ ；而在新树中， T_1 锁定 $A \rightarrow A-B \rightarrow B \rightarrow B-C \rightarrow C$ ， T_2 可锁定 $A \rightarrow A-B \rightarrow B \rightarrow B-D \rightarrow D$ ，由于 $A-B$ 和 $B-D$ 是不同虚拟节点，两个事务可并行执行。

结论：通过插入虚拟节点，树协议的锁粒度更细，锁的释放更加灵活，从而提高了事务的并发性。

问题4：多粒度锁定中的隐式锁定与显式锁定

在多粒度锁定中，隐式锁定和显式锁定有什么区别？

回答：

多粒度锁定的基本概念：允许事务在不同粒度（如数据库、关系、元组等）上获取锁，通过意向锁（Intent Lock）来协调不同粒度的锁。

显式锁定（Explicit Lock）：事务直接请求并获取的锁，在锁表中有明确记录。事务必须显式发出锁请求（如 LOCK TABLE r IN SHARE MODE），锁表中存在该锁的明确条目，可直接检查和释放。例如，事务 T 获取关系 r 的共享锁（S锁）就是显式锁定。

隐式锁定（Implicit Lock）：通过多粒度锁定协议间接获得的锁，在锁表中可能没有明确记录。它基于意向锁的传递性自动获得，锁表中可能无直接记录但可通过上层节点锁信息推导，不需要事务显式请求。例如，事务 T 获取关系 r 的共享锁后，会隐式获得 r 中所有元组的共享锁。

主要区别：显式锁定由事务显式请求获取，锁表中有明确记录，可针对任何粒度，释放方式为显式或自动；隐式锁定则通过意向锁自动获得，锁表中可能无直接记录，通常针对细粒度（如下层节点），随上层显式锁释放而自动释放。

重要性：隐式锁定减少了锁表大小和管理开销，同时通过意向锁机制保证多粒度锁定的正确性，提高系统性能和并发性。

问题5：为事务添加两阶段锁并分析死锁

考虑以下两个事务：

```
T34:
read(A);
read(B);
if A = 0 then B := B + 1;
write(B).

T35:
read(B);
read(A);
if B = 0 then A := A + 1;
write(A).
```

为两个事务添加锁定（lock）和解锁（unlock）的指令，使事务 T34 和 T35 遵循两阶段锁协议(Two-PhaseLocking Protocol)。说明这样会不会导致死锁。

回答：

两阶段锁协议下的事务修改：

根据2PL的要求，每个事务分为增长阶段（获取锁）和缩减阶段（释放锁）。

正确的T34两阶段锁版本（避免锁升级问题）：

```
T34:
lock-S(A);      // 增长阶段：获取A的共享锁
read(A);
```

```
lock-X(B);          // 增长阶段：直接获取B的排他锁（因为可能需要更新）
read(B);
if A = 0 then {
    B := B + 1;
    write(B);
}
unlock(A);          // 缩减阶段：释放A的共享锁
unlock(B);          // 缩减阶段：释放B的排他锁
```

T35的两阶段锁版本：

```
T35:
lock-S(B);          // 增长阶段：获取B的共享锁
read(B);
lock-X(A);          // 增长阶段：直接获取A的排他锁（因为可能需要更新）
read(A);
if B = 0 then {
    A := A + 1;
    write(A);
}
unlock(B);          // 缩减阶段：释放B的共享锁
unlock(A);          // 缩减阶段：释放A的排他锁
```

死锁分析：

考虑以下并发执行场景：

```
T34: lock-S(A) → read(A)
T35: lock-S(B) → read(B)
T34: lock-X(B) → 等待（因为T35已持有B的S锁）
T35: lock-X(A) → 等待（因为T34已持有A的S锁）
```

在这个场景中，T34和T35互相等待对方释放锁，形成了死锁。

原因：死锁源于事务获取锁的顺序不一致：T34先获取A的锁再获取B的锁，而T35先获取B的锁再获取A的锁，这种循环等待的锁获取模式是死锁的典型原因。

避免死锁的方法：可通过统一锁获取顺序解决，即所有事务按相同顺序获取锁（如先A后B）；也可使用意向锁机制减少锁冲突；数据库系统还可通过定期检测死锁并回滚事务来恢复，或采用超时机制，若事务等待锁时间超过阈值则自动回滚并重执行。

结论：在两阶段锁协议下，如果事务获取锁的顺序不一致，可能会导致死锁。因此，需要额外的策略来避免死锁的发生。